Họ và tên: Đoàn Anh Quân

MSSV: 23110111

BÀI THỰC HÀNH DATA MINING TUẦN 1

Mục 2:

- Import các thư viện cần thiết và đọc dữ liệu từ file "arrhythmia.data":

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn import preprocessing

df = pd.read_csv('D:\\data_mining\\tuan1\\arrhythmia.data', delimiter=',', header= None)
print(df)
✓ 0.0s
```

- Thay thế các dữ liệu dạng "?" thành NaN để dễ xử lý. Kiểm tra xem các giá trị missing của data bằng df.isnull().values.any() thay vì df.isnull().sum() để dễ dàng biết được hơn thay vì phải xem từng cột:

```
df.replace('?', np.nan, inplace=True)
#Kiểm tra các giá trị missing
print(df.isnull().values.any())
✓ 0.0s

True
```

- Xử lý các dữ liệu missing và kiểm tra lại:

```python
#Xử lý các dữ liệu thiếu
imputer = SimpleImputer(strategy='mean')
df = pd.DataFrame(imputer.fit_transform(df))
print(df.isnull().values.any())
```

✓ 0.0s

```
False
```

- Kiểm tra xem có các giá trị giống nhau không:

```python
#Kiểm tra giá trị giống nhau
print(df.duplicated().sum())
```

✓ 0.0s

```
0
```

- Sử dụng RobustScaler để xử lý những giá trị ngoại lai:

```python
#Xử lý các giá trị ngoại lai
array = df.values
print(array)

###sử dụng RobustScaler để loại bỏ những giá trị ngoại lai
scaler = preprocessing.RobustScaler()
df_scale = pd.DataFrame(scaler.fit_transform(df))
print('Loai bo cac gia tri ngoai lai:\n', df_scale)
```

✓ 0.0s

```
[[ 75.    0.  190.  ...  23.3  49.4   8. ]
 [ 56.    1.  165.  ...  20.4  38.8   6. ]
 [ 54.    0.  172.  ...  12.3  49.   10. ]
 ...
 [ 36.    0.  166.  ... -44.2 -33.2   2. ]
 [ 32.    1.  155.  ...  25.   46.6   1. ]
 [ 78.    1.  160.  ...  21.3  32.8   1. ]]
Loai bo cac gia tri ngoai lai:
           0    1    2     3         4         5         6         7    \
0    1.272727 -1.0  2.6  0.60  0.357143  1.090909  0.117647  0.387097
1    0.409091  0.0  0.1 -0.20 -0.357143  0.515152  1.000000 -0.419355
2    0.318182 -1.0  0.8  1.35  3.714286  0.181818  0.558824  0.741935
3    0.363636 -1.0  1.1  1.30  1.000000  1.363636  0.382353  0.548387
4    1.272727 -1.0  2.6  0.60  0.142857  0.727273 -0.205882  0.483871
..        ...  ...  ...   ...       ...       ...       ...       ...
447  0.272727  0.0 -0.4  0.10 -0.428571  1.272727  0.441176 -0.258065
448 -0.454545 -1.0  2.6  0.85  1.000000 -0.606061 -0.176471  1.258065
449 -0.500000 -1.0  0.2  0.00  1.571429  0.575758 -0.058824  1.032258
450 -0.681818  0.0 -0.9 -0.65  0.500000 -1.545455  0.558824  1.806452
451  1.409091  0.0 -0.4  0.10 -0.500000 -0.909091 -0.088235 -0.774194

          8         9    ...       270       271       272  273  274  \
0    1.304348 -0.899598  ...  0.000000  0.043478  0.095238  0.0  0.0
1   -2.260870 -0.240964  ...  0.000000 -0.065217  0.523810  0.0  0.0
2    0.478261  0.899598  ...  0.000000  0.152174 -0.619048  0.0  0.0
...
450  0.000000  0.65625  0.480000  0.793213  0.0
451  0.000000  0.15625  0.222609  0.207847  0.0

[452 rows x 280 columns]
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

- Rời rạc hóa mỗi thuộc tính số thành 10 khoảng

```python
#Áp dụng cho toàn bộ các cột của dữ liệu
df_equi_width = df.copy()
df_equi_depth = df.copy()

for col in df.columns:
        #10 Equi-width ranges
        df_equi_width[f'col_{col}_width'] = pd.cut(df[col], bins=10)

        #10 Equi-depth ranges
        df_equi_depth[f'col_{col}_depth'] = pd.qcut(df[col], q=10, duplicates='drop')

print("Equi-width (5 cột đầu):\n", df_equi_width.iloc[:, -5:].head())
print("\nEqui-depth (5 cột đầu):\n", df_equi_depth.iloc[:, -5:].head())
```
✓ 0.6s

```
Equi-width (5 cột đầu):
    col_275_width col_276_width col_277_width  col_278_width col_279_width
0    (0.8, 1.12]    (2.4, 3.6]  (22.3, 35.6]  (38.65, 54.1]    (7.0, 8.5]
1   (0.16, 0.48]    (1.2, 2.4]   (9.0, 22.3]  (38.65, 54.1]    (5.5, 7.0]
2   (0.16, 0.48]    (2.4, 3.6]   (9.0, 22.3]  (38.65, 54.1]   (8.5, 10.0]
3   (0.16, 0.48]    (2.4, 3.6]  (22.3, 35.6]  (54.1, 69.55]  (0.985, 2.5]
4  (-0.16, 0.16]    (3.6, 4.8]  (22.3, 35.6]  (54.1, 69.55]    (5.5, 7.0]

Equi-depth (5 cột đầu):
     col_275_depth col_276_depth  col_277_depth   col_278_depth col_279_depth
0      (0.8, 0.9]    (2.8, 6.0]   (22.0, 24.6]   (44.98, 52.9]    (4.0, 8.0]
1     (0.11, 0.3]    (1.9, 2.3]   (18.1, 22.0]  (37.94, 44.98]    (4.0, 8.0]
2     (0.11, 0.3]    (2.8, 6.0]    (9.6, 12.7]   (44.98, 52.9]   (8.0, 10.0]
3      (0.3, 0.4]    (2.3, 2.8]  (27.98, 34.8]   (52.9, 115.9]  (0.999, 2.0]
4  (-0.801, 0.11]    (2.8, 6.0]  (24.6, 27.98]   (52.9, 115.9]    (4.0, 8.0]
```

Mục 3:

- Import các thư viện cần thiết, chuẩn bị tên cột và đọc dữ liệu từ file “clean2.data”:

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn import preprocessing
from sklearn.decomposition import PCA

feature_names = [f'f{i}' for i in range(1, 167)]
column_names = ['molecule_name', 'conformation_name'] + feature_names + ['class']

df = pd.read_csv('D:\\data_mining\\tuan1\\clean2.data', header= None, names=column_names)
print(df)
✓ 0.1s
```

```
      molecule_name conformation_name  f1   f2   f3   f4   f5   f6   f7   f8  \
0          MUSK-211           211_1+1  46 -108  -60  -69 -117   49   38 -161
1          MUSK-211          211_1+10  41 -188 -145   22 -117   -6   57 -171
2          MUSK-211          211_1+11  46 -194 -145   28 -117   73   57 -168
3          MUSK-211          211_1+12  41 -188 -145   22 -117   -7   57 -170
4          MUSK-211          211_1+13  41 -188 -145   22 -117   -7   57 -170
...             ...               ...  ..  ...  ...  ...  ...  ...  ...  ...
6593  NON-MUSK-jp13          jp13_2+5  51 -123  -23 -108 -117  134 -160   82
6594  NON-MUSK-jp13          jp13_2+6  44 -104  -19 -105 -117  142 -165   68
6595  NON-MUSK-jp13          jp13_2+7  44 -102  -19 -104 -117   72 -165   65
6596  NON-MUSK-jp13          jp13_2+8  51 -121  -23 -106 -117   63 -161   79
6597  NON-MUSK-jp13          jp13_2+9  51 -122  -23 -106 -117  190 -161   80

      ...  f158  f159  f160  f161  f162  f163  f164  f165  f166  class
0     ...  -308    52    -7    39   126   156   -50  -112    96    1.0
1     ...   -59    -2    52   103   136   169   -61  -136    79    1.0
2     ...  -134  -154    57   143   142   165   -67  -145    39    1.0
3     ...   -60    -4    52   104   136   168   -60  -135    80    1.0
4     ...   -60    -4    52   104   137   168   -60  -135    80    1.0
...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...    ...
6593  ...   -66   164   -14   -29   107   171   -44  -115   118    0.0
6594  ...   -51   166    -9   150   129   158   -66  -144    -5    0.0
6595  ...    90   117    -8   150   130   159   -66  -144    -6    0.0
6596  ...    86    99   -14   -31   106   171   -44  -116   117    0.0
6597  ...    40   124   -14   -30   107   171   -44  -115   118    0.0

[6598 rows x 169 columns]
```

- Kiểm tra các giá trị missing và giá trị trùng lặp:

```
#2 - Kiểm tra các giá trị missing
df.replace('?', np.nan, inplace=True)
print(df.isnull().values.any())
```

✓ 0.0s

```
False
```

```
#Kiểm tra giá trị giống nhau
print(df.duplicated().sum())
```

✓ 0.0s

```
0
```

- Chọn X là các cột số, chuẩn hóa Standard và mảng về lại DataFrame:

```
X = df.drop(['molecule_name', 'conformation_name', 'class'], axis=1)
#- chuẩn hóa dữ liệu bằng phương pháp z-score(Standard)
scaler = preprocessing.StandardScaler()
array = scaler.fit_transform(X)

df_standard = pd.DataFrame(array, columns=X.columns)

print(df_standard.head())
```

✓ 0.0s

- Áp dụng PCA:

```
#Áp dụng PCA
pca = PCA()
pca.fit(df_standard)
```

✓ 0.0s

PCA

▼ Parameters

n_components	None
copy	True
whiten	False
svd_solver	'auto'
tol	0.0
iterated_power	'auto'
n_oversamples	10
power_iteration_normalizer	'auto'
random_state	None

- Tìm các trị riêng, vector riêng:

```python
eigenvalues = pca.explained_variance_
eigenvectors = pca.components_

for i, val in enumerate(eigenvalues):
    print(f"Thành phần chính {i+1}:")
    print(f"Eigenvalue (Trị riêng): {val:.4f}")
    print(f"Eigenvector (Vector riêng): {eigenvectors[i]}")
    print("-" * 30)
```

✓ 0.1s

```
Thành phần chính 1:
Eigenvalue (Trị riêng): 48.5969
Eigenvector (Vector riêng): [-0.00209029 -0.07861939 -0.11635903  0.09093466  0.00447044 -0.04208069
  0.1291817  -0.03589637  0.12507385 -0.0965009   0.01001111  0.03422102
 -0.0241283  -0.02362391 -0.01263891 -0.00595319  0.11838846 -0.01035162
  0.01571339 -0.04413033  0.1034642   0.13118846  0.12663114 -0.09422176
 -0.11924004  0.11698988  0.08463097  0.07363604  0.05315825 -0.11631681
 -0.00109232 -0.01223101  0.08736826 -0.10072166  0.1090668  -0.01665723
  0.03099184  0.08487557 -0.1168716   0.07996433 -0.11407075 -0.01304781
 -0.06093758 -0.00459002 -0.00984955  0.00836212  0.0268031   0.09942624
 -0.0257963  -0.08820216  0.06715848  0.13013847  0.13009622  0.00873972
 -0.0700594   0.07239032  0.12372576 -0.03286856  0.01719912 -0.11055741
  0.08788373 -0.0427868   0.01595499 -0.11157337  0.0948442  -0.00809826
 -0.00661959 -0.03896541 -0.05193892 -0.03806689 -0.11651146 -0.11433733
  0.00322992 -0.00995461 -0.03839837  0.00461827  0.12388783 -0.02280412
  0.03563347  0.00694298  0.12162369  0.13075667  0.09581106 -0.02580435
 -0.12102817  0.12629465 -0.04417801  0.04976655  0.0805741  -0.11024172
 -0.02928875 -0.00380969  0.11242655 -0.02562724  0.03434498 -0.02343775
  0.00316932  0.12807111 -0.12534451  0.12156474 -0.10738608 -0.00062471
 -0.00865103  0.00568218 -0.0103443  -0.02902687  0.04263055  0.09609301
 -0.03614858 -0.00878164 -0.00949343  0.12248309  0.13208626  0.13330524
 -0.1172947  -0.06536125 -0.11441337  0.11972527  0.12644258  0.08566335
 -0.11981202 -0.07078749 -0.09435431  0.03997485 -0.01456602 -0.02399716
 -0.09123638 -0.11106908 -0.06694909  0.10215797  0.02355469  0.00318372
  0.0610299   0.00622925 -0.03572764 -0.07134634  0.10627556 -0.06606883
...
 -9.49867436e-04  4.45255100e-04 -6.43367211e-04  3.22787781e-03
 -2.04330499e-04  2.80960656e-04 -1.78061764e-03  3.65908532e-04
 -9.64596094e-04  8.07422744e-04]
------------------------------
```